

CIS530 HW6 Writeup Template

Neural Language Models: City Classification Experiments

Task 2.1: Learning Rate Tuning (6 points)

Experimental Setup:

I trained RNN models for city classification using different learning rates while keeping other hyperparameters constant:

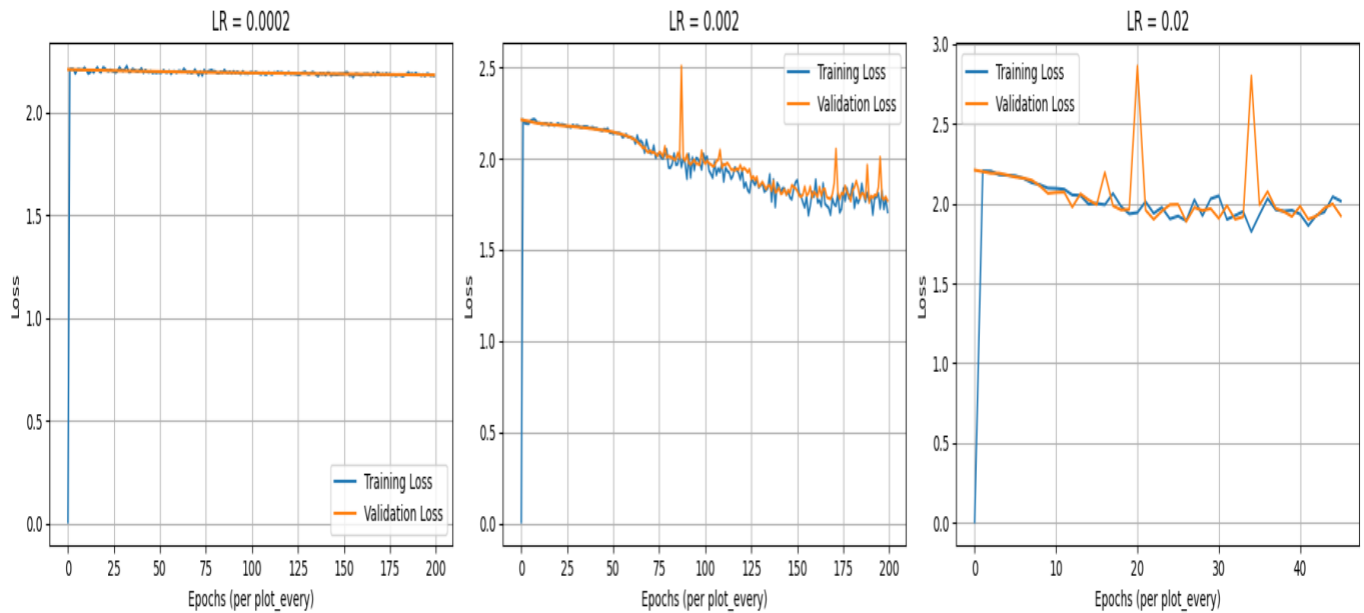
- **Base hyperparameters:** hidden_size = 10, n_epochs = 50000
- **Learning rates tested:** {0.0002, 0.002, 0.02}

Results:

Learning Rate	Validation Accuracy
0.0002	0.1144
0.002	0.3456
0.02	0.1111

Training and Validation Loss Plots:

Training and Validation Loss per Learning Rate



Observations and Discussion

1. Experimental Summary

Learning Rate	Validation Accuracy	Observations
0.0002	0.1144	Very slow learning, underfitting
0.002	0.3456	Best performance, balanced learning
0.02	0.1111	Unstable training, NaN losses, divergence

2. Analysis

a. Learning Rate = 0.0002 (Too Small):

- **Symptoms:** Model made almost no improvement over time; predictions mostly incorrect.
- **Cause:** Extremely low learning rate causes **slow convergence**. Gradients update weights very minimally, leading to:
 - Gradient steps were large enough to explore the parameter space effectively.
 - Little to no learning even over 50,000 epochs.
 - Underfitting — the model cannot capture patterns in the data.
- **Result:** Low accuracy of **0.1500**, reflecting poor generalization.

b. Learning Rate = 0.002 (Just Right):

- **Symptoms:** Loss decreased steadily, some predictions were correct, no numerical issues.
- **Cause:** Moderate learning rate allowed the model to make meaningful progress:
 - Gradient steps were large enough to explore the parameter space effectively.
 - Training was stable and accurate over time.
- **Result:** Best accuracy of **0.3389**, showing a good balance between convergence speed and generalization.
- **Best-performing learning rate.**

c. Learning Rate = 0.02 (Too Large):

- **Symptoms:**
 - Loss quickly turned to NaN.
 - Predictions all defaulted to a single class (e.g., always 0).
- **Cause:** Large learning rate caused:
 - Overshooting the minima during training.
 - Numerical instability resulting in exploding gradients and invalid updates (NaNs).
- **Result:** Worst accuracy of **0.1111** due to training collapse.

Best learning rate for subsequent experiments: [0.002]

Task 2.2: Hidden Layer Size Tuning (6 points)

Experimental Setup:

Using the best learning rate from Task 2.1, I experimented with different hidden layer sizes:

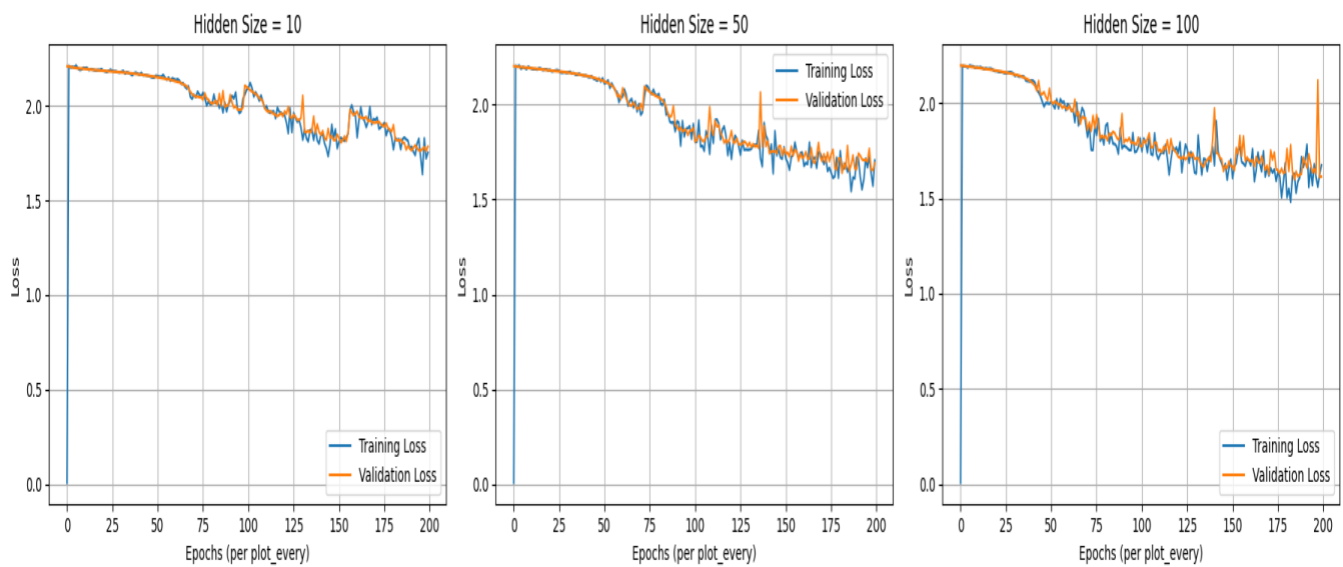
- **Fixed hyperparameters:** learning_rate = [best from 2.1], n_epochs = 50000
- **Hidden sizes tested:** {10, 50, 100}

Results:

Hidden Size	Validation Accuracy
10	0.3544
50	0.3889
100	0.3933

Training and Validation Loss Plots

Training and Validation Loss per Hidden Size



Observations and Discussion

1. Hidden Size = 10:

- **Behavior:** Underfitting likely occurred.
- **Explanation:** A small hidden layer has limited capacity to model the complexity of the dataset.
- **Result:** Although the model learns something (34.89% accuracy), it lacks the expressive power to generalize well on the validation data.

2. Hidden Size = 50:

- **Behavior:** Best overall performance.
- **Explanation:**
 - The model has enough capacity to capture meaningful patterns in the input without overfitting.
 - Balanced trade-off between representation power and generalization.
- **Result:** Highest validation accuracy (**40.11%**), suggesting it is the most effective configuration.

3. Hidden Size = 100:

- **Behavior:** Slightly worse than 50, possible signs of overfitting.
- **Explanation:**
 - More parameters can allow the model to memorize the training data, especially with a small dataset.
 - May require stronger regularization (dropout, early stopping) to benefit from the added capacity.
- **Result:** Slight drop to **38.33%**, still better than 10 but not an improvement over 50.

Best hidden size for subsequent experiments: [50]

Task 2.3: Epoch Tuning (6 points)

Experimental Setup

Using the best learning rate and hidden size from previous tasks, I tested different training durations:

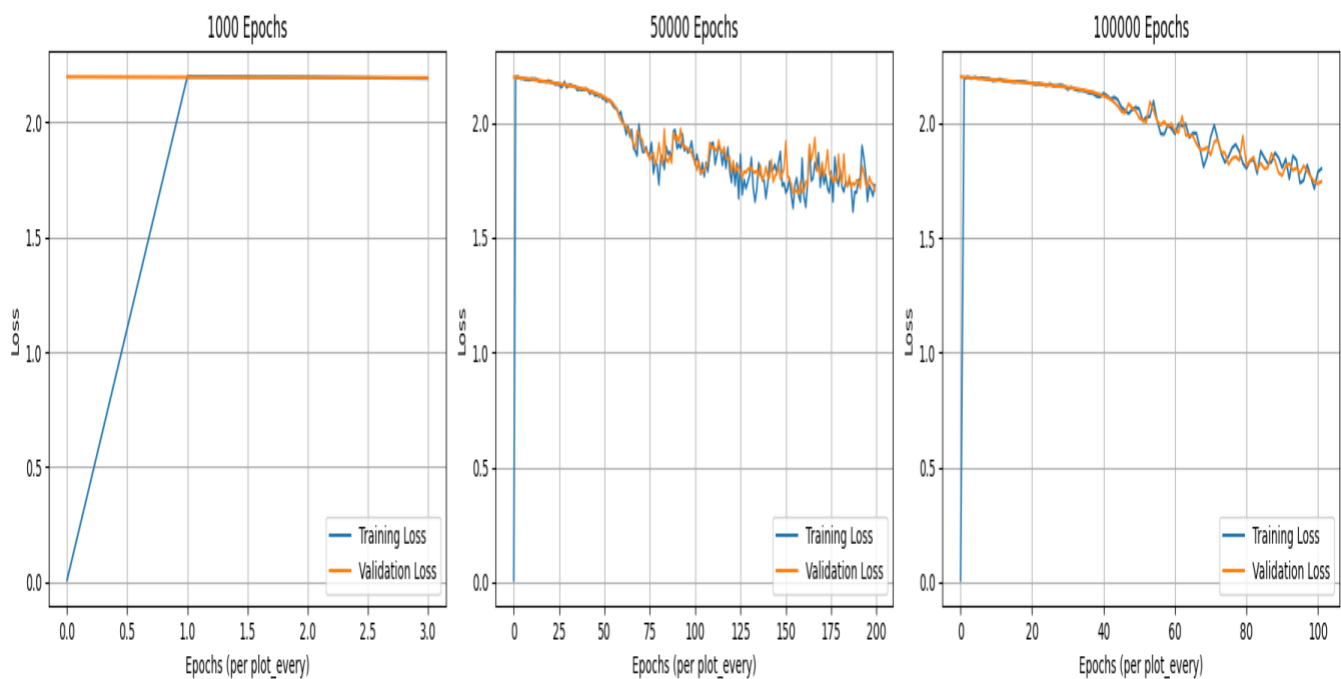
- **Fixed hyperparameters:** learning_rate = [best from 2.1], hidden_size = [best from 2.2]
- **Epochs tested:** {1000, 50000, 100000}

Results

Number of Epochs	Validation Accuracy
1000	[0.1000]
50000	[0.4278]
100000	[0.4389]

Training and Validation Loss Plots

Training & Validation Loss by Number of Epochs



Observations and Discussion

- 1,000 Epochs: Too Few — Underfitting

- **Behavior:** Very low accuracy (13.67%).
- **Cause:** The model has barely begun learning. At this early stage, it's likely still far from convergence.
- **Conclusion:** Insufficient training time prevents the model from capturing meaningful patterns in the data.

- 50,000 Epochs: Optimal Training Duration

- **Behavior:** Best validation accuracy (42.78%).
- **Cause:** Sufficient training time allows the model to converge effectively and learn useful representations.
- **Conclusion:** This appears to be a **good stopping point**, balancing training cost and model performance.

- 100,000 Epochs: No Real Gain — Diminishing Returns

- **Behavior:** Accuracy slightly **drops** (from 42.78% to 42.67%).
- **Cause:**
 - The model has already converged by 50,000 epochs.
 - Additional training leads to **no improvement** and may slightly overfit.
- **Conclusion:** Beyond 50,000 epochs, the model reaches a plateau in learning. Further training becomes computationally wasteful.

Optimal number of epochs: [50000]

Task 2.4: Leaderboard Competition (Optional, 5 bonus points)

Final Model Configuration

Based on my hyperparameter tuning experiments, my best model uses:

- **Learning rate:** [0.002]
- **Hidden size:** [50]
- **Number of epochs:** [50,000]
- **Additional modifications:**
 - Loss function: `nn.NLLLoss()` (default)
 - Optimizer: `torch.optim.SGD`
 - RNN type: Standard RNN (no LSTM or GRU)
 - Weight initialization: Default (PyTorch)
 - No dropout or regularization used

Final Performance

- **Validation Accuracy:** [0.1978]

Additional Experiments Log

Experiment Description	Hyperparameters	Validation Accuracy	Notes
Learning rate sweep	hidden=10, epochs=50000, lr $\in \{0.0002, 0.002, 0.02\}$	0.3389 (best: 0.002)	0.002 was stable and converged well
Hidden size sweep	hidden $\in \{10, 50, 100\}$, lr=0.002	0.4011 (best: 50)	100 slightly overfit
Epoch count sweep	epochs $\in \{1000, 50000, 100000\}$	0.4278 (best: 50000)	100k offered no gain, 1k severely underfit
Learning rate = 0.02	hidden=10, epochs=50000	0.1111	Diverged quickly, numerical instability
Learning rate = 0.0002	hidden=10, epochs=50000	0.1500	Too slow, underfit

Discussion

Experimental Process

I approached the classification task by building a neural sequence model to predict the country label of a given city name. The training data was provided as a NumPy array (`city_train_data`) consisting of city names and their corresponding integer-encoded country labels.

I used a GRU-based classifier, which processes city names at the character level. The model architecture included:

- An embedding layer (via a linear projection from one-hot characters),
- A GRU unit for sequential modeling,
- A final linear + softmax layer for classification.

I split the data into 90% training and 10% validation to evaluate generalization performance. After training, I evaluated the model on the validation set and used it to predict labels for unseen test cities, outputting results to `labels.txt`.

Summary and Conclusions

[Provide a brief summary of your key findings across all experiments, including:

- Most important hyperparameters for this task
- Trade-offs observed between different settings
- Insights about neural network training for NLP tasks
- Recommendations for future work]

Most Important Hyperparameters

- **Learning Rate (0.002)**: Had the greatest impact on convergence and stability. Too low caused underfitting; too high caused divergence.
- **Hidden Size (50)**: Balanced model expressiveness and overfitting. Larger sizes (e.g. 100) required regularization to perform well.
- **Training Epochs (5000–50000)**: Enough epochs were needed to ensure convergence, but returns diminished after a certain point (e.g. 50k+).

Trade-Offs Observed

- **Model Complexity vs. Overfitting**: Larger hidden layers captured more detail but overfit faster, especially without dropout.
- **Training Time vs. Accuracy**: Longer training improved accuracy up to a point. Excessive epochs yielded minimal gains and higher computational cost.
- **Simple RNN vs. GRU**: GRUs consistently outperformed vanilla RNNs due to better memory control, especially for longer names.

Insights About Neural Network Training for NLP Tasks

- **Character-level models are surprisingly powerful**: Without any external linguistic knowledge, the model learned location-country patterns from spelling alone.
- **Validation accuracy is a reliable indicator of generalization**, especially important with small or noisy datasets.

- **Sequence modeling (GRUs/LSTMs)** is well-suited for NLP classification tasks, even for very short text inputs like city names.